



HeeraMon

API Specification

API Specification Document

Date of Preparation: September 30, 2026

Document ID	HM-DOC-10
Title	API Specification
Classification	API Specification Document
Status	Draft
Version	1.0
Owner	Product / Engineering
Audience	All project contributors
Related Documents	Technical Architecture (HM-DOC-07), SRS (HM-DOC-02)

Contents

1 Purpose	2
2 API Principles	2
3 Authentication	2
4 Core Endpoints	3
4.1 Authentication	3
4.2 Conversations	3
4.3 Preferences	3
4.4 Jobs	3
4.5 Matches	4
4.6 Notifications	4
4.7 Applications	4
5 Background Operations	5
6 Request and Response Design	5
7 Pagination	5
8 Validation	6
9 Error Handling	6
10 Rate Limiting	6
11 API and Agent Boundary	7
12 Boundaries	7
13 Success Criteria	7
14 Document Control	8

1 Purpose

This document defines the primary API contract for HeeraMon.

The API connects the frontend with backend services and provides controlled access to user data, conversations, jobs, preferences, matches, notifications, and application workflows.

The API should remain versioned, authenticated, validated, and independent of the frontend implementation.

2 API Principles

The API should:

- use predictable resource-oriented endpoints;
- validate all incoming data;
- enforce authentication and authorization server-side;
- return structured responses;
- provide meaningful error responses;
- avoid exposing internal implementation details;
- keep LLM output behind controlled backend logic;
- support future API versioning.

Proposed base path:

```
/api/v1
```

3 Authentication

Authentication should be handled through the chosen authentication provider, with the backend validating the authenticated user's identity.

Every user-specific endpoint must verify that the requested resource belongs to the authenticated user.

Authentication should support frictionless providers where practical, such as:

- Google;
- GitHub;
- email-based authentication.

The API must never trust a user-supplied `user_id` as proof of authorization.

4 Core Endpoints

4.1 Authentication

Authentication may primarily be handled by the authentication provider rather than custom application endpoints.

4.2 Conversations

```
GET    /conversations
POST   /conversations
GET    /conversations/{conversation_id}
POST   /conversations/{conversation_id}/messages
```

Responsibilities:

- retrieve conversation history;
- create conversations;
- send messages;
- return HeeraMon responses.

The backend controls agent/tool execution.

4.3 Preferences

```
GET    /preferences
POST   /preferences
PATCH /preferences/{preference_id}
DELETE /preferences/{preference_id}
```

Responsibilities:

- retrieve current preferences;
- create/update preferences;
- allow user corrections;
- manage learned preferences.

Important preference changes should be validated and persisted by the backend.

4.4 Jobs

```
GET    /jobs
GET    /jobs/{job_id}
```

Responsibilities:

- retrieve available jobs;

- retrieve detailed job information;
- support filtering and pagination.

Users should not directly control internal ingestion operations through public endpoints.

4.5 Matches

```
GET      /matches
GET      /matches/{match_id}
POST     /matches/{match_id}/decision
```

Responsibilities:

- retrieve relevant opportunities;
- inspect match explanations;
- record user decisions.

4.6 Notifications

```
GET      /notifications
PATCH   /notifications/{notification_id}
```

Responsibilities:

- retrieve notifications;
- mark notifications as read;
- expose notification status.

4.7 Applications

```
POST     /applications
GET      /applications
GET      /applications/{application_id}
PATCH   /applications/{application_id}
POST     /applications/{application_id}/prepare
POST     /applications/{application_id}/approve
POST     /applications/{application_id}/cancel
```

Responsibilities:

- create application workflows;
- prepare application information;
- retrieve application state;
- allow editing;
- handle explicit approval;
- cancel applications.

The approve operation must be enforced by backend logic as the human-approval boundary.

5 Background Operations

Internal background workers may use private service interfaces for:

- job ingestion;
- job normalization;
- deduplication;
- matching;
- notification generation;
- scheduled searches;
- agent tasks.

These operations do not need to be exposed as public user-facing endpoints.

6 Request and Response Design

Responses should use consistent structured formats.

Example:

```
{
  "data": {},
  "error": null,
  "meta": {}
}
```

Successful responses should provide the requested resource or operation result.

Errors should provide enough information for the frontend to recover without exposing sensitive internal details.

7 Pagination

Collection endpoints should support pagination.

Examples:

```
GET /jobs?page=1&limit=20
GET /matches?page=1&limit=20
GET /notifications?page=1&limit=20
```

The API should avoid returning unnecessarily large datasets.

Cursor-based pagination may be introduced later where required by scale.

8 Validation

The backend should validate:

- request structure;
- field types;
- allowed values;
- resource ownership;
- application state transitions;
- required fields.

LLM-generated structured output must also be validated before being persisted or used for consequential actions.

9 Error Handling

The API should use appropriate HTTP status codes and consistent error structures.

Common cases include:

```
400 Invalid request
401 Unauthenticated
403 Unauthorized
404 Resource not found
409 Invalid state/conflict
422 Validation error
429 Rate limited
500 Internal server error
503 External service unavailable
```

Errors should be safe for users while detailed internal information is logged server-side.

10 Rate Limiting

Rate limiting should protect:

- public endpoints;
- authentication-related operations;
- expensive agent operations;
- job searches;
- application workflows.

Limits should be configurable rather than hard-coded throughout the application.

11 API and Agent Boundary

The LLM agent should not receive unrestricted API or database access.

Instead:

Agent → Approved Tool → Backend Service → Database / External Service

Tools should expose only the operations required for the current task.

This ensures that agent reasoning remains separated from application authorization and business logic.

12 Boundaries

The initial release does not require:

- a public developer API;
- third-party API keys for users;
- GraphQL;
- microservice-to-microservice APIs;
- real-time WebSocket infrastructure unless required by the implementation;
- extensive API documentation portals.

OpenAPI documentation should nevertheless be generated for the internal REST API.

13 Success Criteria

The API is successful when it can securely support:

1. Authentication and user access.
2. Conversations with HeeraMon.
3. Preference management.
4. Job retrieval.
5. Match retrieval and decisions.
6. Notifications.
7. Application preparation and review.
8. Explicit application approval.
9. Background service operations.

Core principle: The API is the controlled boundary between HeeraMon's interface, agent, persistent data, and external systems.

14 Document Control

Version	Date	Author / Role	Changes
1.0	September 30, 2026	Product / Engineering	Initial release

Approval: This document defines the API contract for HeeraMon. All subsequent design and implementation work must remain consistent with the principles stated herein.

HeeraMon flies to help people find job that fits not only their career, but the life they want to live.



Saadman Sakib
Product Engineer